# Chapter 6: Multiple Linear Regression

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

### # JT88

### # This code cell imports necessary Python packages.

In [66]:
```
pip install mlba
```

```
Requirement already satisfied: mlba in /opt/anaconda3/lib/python3.13/site-pa
ckages (2.0.2)
Note: you may need to restart the kernel to use updated packages.
```

"mlxtend" means Machine Learning Extensions - it's a Python library that provides additional tools for machine learning, such as model selection, feature selection, etc.

In [67]:
```
pip install mlxtend
```

```
Requirement already satisfied: mlxtend in /opt/anaconda3/lib/python3.13/site
-packages (0.23.4)
Requirement already satisfied: scipy>=1.2.1 in /opt/anaconda3/lib/python3.1
3/site-packages (from mlxtend) (1.15.3)
Requirement already satisfied: numpy>=1.16.2 in /opt/anaconda3/lib/python3.1
3/site-packages (from mlxtend) (2.1.3)
Requirement already satisfied: pandas>=0.24.2 in /opt/anaconda3/lib/python3.
13/site-packages (from mlxtend) (2.2.3)
Requirement already satisfied: scikit-learn>=1.3.1 in /opt/anaconda3/lib/pyt
hon3.13/site-packages (from mlxtend) (1.6.1)
Requirement already satisfied: matplotlib>=3.0.0 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from mlxtend) (3.10.0)
Requirement already satisfied: joblib>=0.13.2 in /opt/anaconda3/lib/python3.
13/site-packages (from mlxtend) (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/python3.1
3/site-packages (from matplotlib>=3.0.0->mlxtend) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (24.2)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.13/si
te-packages (from matplotlib>=3.0.0->mlxtend) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/py
thon3.13/site-packages (from matplotlib>=3.0.0->mlxtend) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.1
3/site-packages (from pandas>=0.24.2->mlxtend) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.
13/site-packages (from pandas>=0.24.2->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.13/sit
e-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/lib/py
thon3.13/site-packages (from scikit-learn>=1.3.1->mlxtend) (3.5.0)
Note: you may need to restart the kernel to use updated packages.
```

In [68]:

```
import matplotlib.pyplot as plt # for plotting
import mlba
import numpy as np # for numerical data handling
import pandas as pd # for tabular data handling
import statsmodels.formula.api as sm # for statistical modeling
from mlxtend.feature_selection import ExhaustiveFeatureSelector, SequentialF
from sklearn.linear_model import BayesianRidge, Lasso, LassoCV, LinearRegres
from sklearn.model_selection import cross_validate, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
%matplotlib inline
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.linear_model import LinearRegression
```

# MLR - Predicting the Price of Used Toyota Corolla Cars

A large Toyota car dealership offers purchasers of new Toyota cars the option to buy their used car as part of a trade-in. In particular, a new promotion promises to pay high prices for used Toyota Corolla cars for purchasers of a new car. The dealer then sells the used cars for a small profit. To ensure a reasonable profit, the dealer needs to be able to predict the price that the dealership will get for the used cars. For that reason, data were collected on all previous sales of used Toyota Corollas at the dealership. The data include the sales price and other information on the car, such as its age, mileage, fuel type, and engine size. A description of each of these variables is given in Table 6.1. A sample of this dataset is shown in Table 6.2. The total number of records in the dataset is 1000 cars (we use the first 1000 cars from the dataset ToyotoCorolla.csv ). After partitioning the data into training (60%) and holdout (40%) sets, we fit a multiple linear regression model between price (the outcome variable) and the other variables (as predictors) using only the training set.

Handling categorical variables: Notice that the Fuel Type predictor has three categories (Petrol, Diesel, and CNG). We therefore have two dummy variables in the model: Fuel_Type_Petrol (0/1) and Fuel_Type_Diesel (0/1); the third, for CNG (0/1), is redundant given the information on the first two dummies. Including the redundant dummy would cause the regression to fail, since the redundant dummy will be a perfect linear combination of the other two. We use the pandas data frame function get_dummies to create the dummy variables.

## Table 6.3 Fit Linear Regression Model

In [69]:
```
#VB67
# reducing the data frame to the top 1000 rows and select columns for regres
car_df = mlba.load_data('ToyotaCorolla.csv')
car_df = car_df.iloc[0:1000]

predictors = ['Age_08_04', 'KM', 'Fuel_Type', 'HP', 'Met_Color', 'Automatic'
              'Doors', 'Quarterly_Tax', 'Weight']
outcome = 'Price'

# Converts categorical variable Fuel_Type into dummy variables using pd.get_
# "drop_first=True" drops the redundant dummy variable to avoid multicolline
X = pd.get_dummies(car_df[predictors], drop_first=True)
y = car_df[outcome]

# partition data
# Spliting the data into training (60%) and holdout/test (40%) sets, reprodu
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.

                                                          random_state=314)
```

```
# fits a linear regression model to the training set
car_lm = LinearRegression()
car_lm.fit(train_X, train_y)

# printing coefficients
print(pd.DataFrame({'Predictor': X.columns, 'coefficient': car_lm.coef_}))

# printing performance measures (training data)
mlba.regressionSummary(y_true=train_y, y_pred=car_lm.predict(train_X))
```

```
           Predictor  coefficient
0          Age_08_04  -130.792830
1                 KM    -0.018302
2                 HP    64.834495
3          Met_Color    54.282338
4          Automatic   298.009711
5                 CC    -4.251357
6              Doors   -25.888900
7      Quarterly_Tax    18.013228
8             Weight    15.547273
9   Fuel_Type_Diesel  3875.777969
10  Fuel_Type_Petrol  2897.532504

Regression statistics

Mean Error (ME) : 0.0000
Root Mean Squared Error (RMSE) : 1340.1580
Mean Absolute Error (MAE) : 987.7296
Mean Percentage Error (MPE) : -0.9911
Mean Absolute Percentage Error (MAPE) : 8.8094
```

In [70]:
```
#VB67
# showing the dimensions (number of rows and columns) of the DataFrame car_c
car_df.shape
```

Out[70]: (1000, 39)

# Table 6.4 Predicted Price and Errors for 20 Cars in Holdout Set

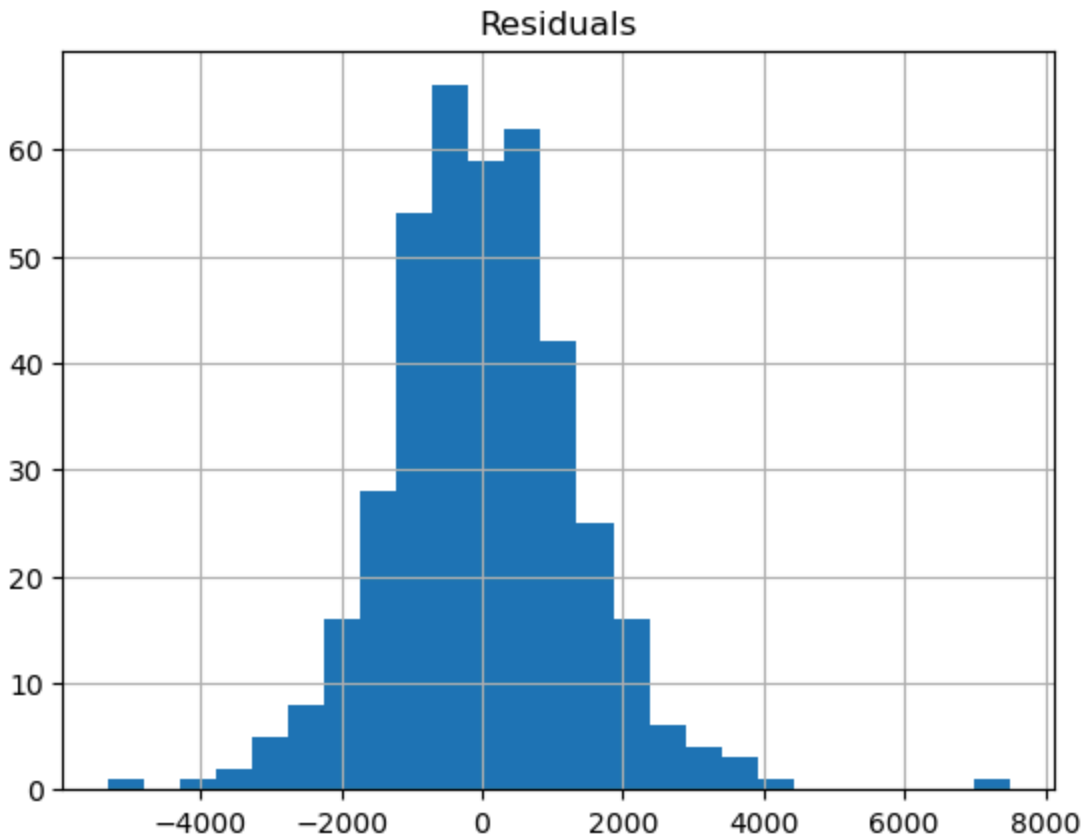
In [71]:
```
#VB67
# Using predict() to make predictions on a new set
car_lm_pred = car_lm.predict(holdout_X)

result = pd.DataFrame({'Predicted': car_lm_pred, 'Actual': holdout_y,
                       'Residual': holdout_y - car_lm_pred})
print(result.head(20))

# printing performance measures (holdout data)
mlba.regressionSummary(y_true=holdout_y, y_pred=car_lm_pred)
```

```
        Predicted  Actual     Residual
764  10073.017338   10950   876.982662
277  12649.704215   11495 -1154.704215
515  10317.854192   10750   432.145808
305  13075.699963   11895 -1180.699963
108  17210.881336   17950   739.118664
392  10627.038857    9950  -677.038857
905   9179.342231    9950   770.657769
507  11015.919573   11500   484.080427
789   9880.554635    8950  -930.554635
260  12124.410668   11950  -174.410668
69   16177.924091   15950  -227.924091
60   16429.726963   14950 -1479.726963
744   7987.909413    7450  -537.909413
584  10731.467539   10500  -231.467539
223  11392.715402   14950  3557.284598
104  19597.277288   19450  -147.277288
335  13441.801213   12500  -941.801213
463  11886.513474    8750 -3136.513474
224  13559.243997   12450 -1109.243997
269  12608.507012   10950 -1658.507012

Regression statistics

Mean Error (ME) : -1.3621
Root Mean Squared Error (RMSE) : 1355.0409
Mean Absolute Error (MAE) : 1028.0884
Mean Percentage Error (MPE) : -1.2047
Mean Absolute Percentage Error (MAPE) : 9.0629
```

# Figure 6.1 HISTOGRAM OF MODEL ERRORS (BASED ON HOLDOUT SET)

The residuals follow the normal probability distribution

In [72]:
```
#VB67
#Plot histogram of residuals (errors) to check distribution and model fit
car_lm_pred = car_lm.predict(holdout_X)
all_residuals = holdout_y - car_lm_pred

pd.DataFrame({'Residuals': all_residuals}).hist(bins=25)
plt.show()
```




## Table 6.5 Cross-Validation in Toyota Corolla Example

In scikit-learn, you can use the cross_validate function to perform cross-validation. When using cross-validation for evaluating performance, we do not need a separate holdout set; instead we apply cross-validation to the entire dataset.

The scoring metrics are negative RMSE and negative MAE. The two metrics are negative because scikit-learn uses a convention where higher scores are better. We therefore need to invert the results by multiplying "-1" because the actual RMSE/MAE is positive.

In [73]:
```
#VB67
from sklearn.model_selection import cross_validate
# Perform 5-fold cross-validation on the training set using Linear Regressic
# - Evaluate model performance with RMSE and MAE

model = LinearRegression()

# - Negatives are used because sklearn expects a "score" to be maximized
scoring = {'neg_RMSE': 'neg_root_mean_squared_error',
           'neg_MAE': 'neg_mean_absolute_error'}
scores = cross_validate(model, train_X, train_y, cv=5, scoring=scoring)

# - Print average cross-validated RMSE and MAE to assess model generalizatic
print(f"CV-RMSE = {- scores['test_neg_RMSE'].mean():.2f}")
print(f"CV-MAE = {- scores['test_neg_MAE'].mean():.2f}")
```

```
CV-RMSE = 1514.51
CV-MAE = 1052.89
```

# Variable Selection in Linear Regression

# Exhaustive Search

# Table 6.6 Exhaustive Search for Reducing Predictors in Toyota Corolla Example

The ExhaustiveFeatureSelector class is part of the mlxtend package. Here, we explore all models with up to 11 predictors, using 5-fold cross-validation, and RMSE as the scoring metric.

EFS includes all possible combinations of 1 to 11 features and evaluates each subset using 5-fold CV RMSE. The best score is the lowest RMSE.

The best model used five predictors: the age of the car (Age_08_04), its mileage (KM), horsepower (HP), number of doors (Doors), and weight (Weight). The RMSE for this model is $1479.

In [76]:
```
#VB67
# Exhaustive feature selection with Linear Regression using 5-fold CV to fir
model = LinearRegression()
efs = ExhaustiveFeatureSelector(model,
            min_features=1, max_features=11,
            cv=5, scoring='neg_root_mean_squared_error',
            print_progress=False, n_jobs=4)
efs = efs.fit(train_X, train_y)

print(f'Best score: {- efs.best_score_:.2f}')
print(f'Best subset (indices): {efs.best_idx_}')
print(f'Best subset (corresponding names):\n{efs.best_feature_names_}')
```

```
Best score: 1479.38
Best subset (indices): (0, 1, 2, 6, 8)
Best subset (corresponding names):
('Age_08_04', 'KM', 'HP', 'Doors', 'Weight')
```

In [79]:
```
#VB67
# Storing cross-validation results for RMSE and MAE with negative sign to ge
cv_exhaustive = {'RMSE': - scores['test_neg_RMSE'].mean(),
                 'MAE': - scores['test_neg_MAE'].mean()}
# Extracting the best feature subset names selected by exhaustive feature se
efs_subset = list(efs.best_feature_names_)
```

In [80]:
```
#VB67
# Generating the best feature subset and corresponding RMSE for each number
important_features = {}
for v in efs.subsets_.values():
```

```
    n = len(v['feature_names'])
    if n not in important_features:
        important_features[n] = v
    elif v['avg_score'] > important_features[len(v['feature_names'])]['avg_s
        important_features[n] = v
for k, v in important_features.items():
    print(f'{k}, RMSE: {- v["avg_score"]:.2f}, Features: {v["feature_names"]
```

```
1, RMSE: 1895.03, Features: ('Age_08_04',)
2, RMSE: 1697.97, Features: ('Age_08_04', 'Weight')
3, RMSE: 1551.82, Features: ('Age_08_04', 'KM', 'Weight')
4, RMSE: 1479.57, Features: ('Age_08_04', 'KM', 'HP', 'Weight')
5, RMSE: 1479.38, Features: ('Age_08_04', 'KM', 'HP', 'Doors', 'Weight')
6, RMSE: 1482.08, Features: ('Age_08_04', 'KM', 'HP', 'Doors', 'Quarterly_Ta
x', 'Weight')
7, RMSE: 1486.71, Features: ('Age_08_04', 'KM', 'HP', 'Met_Color', 'Automati
c', 'Doors', 'Weight')
8, RMSE: 1485.12, Features: ('Age_08_04', 'KM', 'HP', 'Automatic', 'CC', 'Qu
arterly_Tax', 'Fuel_Type_Diesel', 'Fuel_Type_Petrol')
9, RMSE: 1482.00, Features: ('Age_08_04', 'KM', 'HP', 'Automatic', 'CC', 'Do
ors', 'Quarterly_Tax', 'Fuel_Type_Diesel', 'Fuel_Type_Petrol')
10, RMSE: 1487.74, Features: ('Age_08_04', 'KM', 'HP', 'Met_Color', 'Automat
ic', 'CC', 'Doors', 'Quarterly_Tax', 'Fuel_Type_Diesel', 'Fuel_Type_Petrol')
11, RMSE: 1514.51, Features: ('Age_08_04', 'KM', 'HP', 'Met_Color', 'Automat
ic', 'CC', 'Doors', 'Quarterly_Tax', 'Weight', 'Fuel_Type_Diesel', 'Fuel_Typ
e_Petrol')
```

In [23]:
```
#HIGHER accuracy score BETTER the model.
#HIGHER RMSE - BETTER the model.
```

## Popular Subset Selection Algorithms

Three popular iterative search algorithms are forward selection, backward elimination, and stepwise regression.

The mlxtend package also has an implementation of these algorithms. All three algorithms are implemented in the SequentialFeatureSelector class. The direction of the search is controlled by the forward argument. The results of forward selection and backward elimination are shown in Tables 6.7 and 6.8, respectively. Forward leads to the same five-predictor model as the exhaustive search (RMSE=1479), while backward ends with a nine-predictor model with a slightly higher RMSE (1482). In order to perform stepwise regression, we set the floating argument to True as shown in Table 6.9. Again, we end up with the same five-predictor model with RMSE=1479.

## Table 6.7 Forward Selection for Reducing Predictors in Toyota Corolla Example

In [82]:
```
#VB67
# Performing the forward sequential feature selection with Linear Regressior
# Evaluating feature subsets from 1 to 11 features using 5-fold CV and RMSE
```

```
# Selects the subset with the best (lowest) cross-validated RMSE

sfs_forward = SequentialFeatureSelector(model,
            k_features=(1, 11),
            forward=True, floating=False,
            cv=5, scoring='neg_root_mean_squared_error',
            n_jobs=-1)
sfs_forward = sfs_forward.fit(train_X, train_y)

# sfs_forward.subsets_ becomes a dictionary that stores all 11 feaure subset
best_subset = sfs_forward.subsets_[1] #assuming the best subset is the one w
for v in sfs_forward.subsets_.values():
    if v['avg_score'] > best_subset['avg_score']:
        best_subset = v

# Printing best score, indices, and names of selected features
print(f"Best accuracy score: {- best_subset['avg_score']:.2f}")
print(f"Best subset (indices): {best_subset['feature_idx']}")
print(f"Best subset (corresponding names):\n{best_subset['feature_names']}")
```

```
Best accuracy score: 1479.38
Best subset (indices): (0, 1, 2, 6, 8)
Best subset (corresponding names):
('Age_08_04', 'KM', 'HP', 'Doors', 'Weight')
```

In [83]:

```
#VB67
forward_subset = list(best_subset['feature_names'])
```

# Table 6.8 Backward Elimination for Reducing Predictors in Toyota Corolla Example

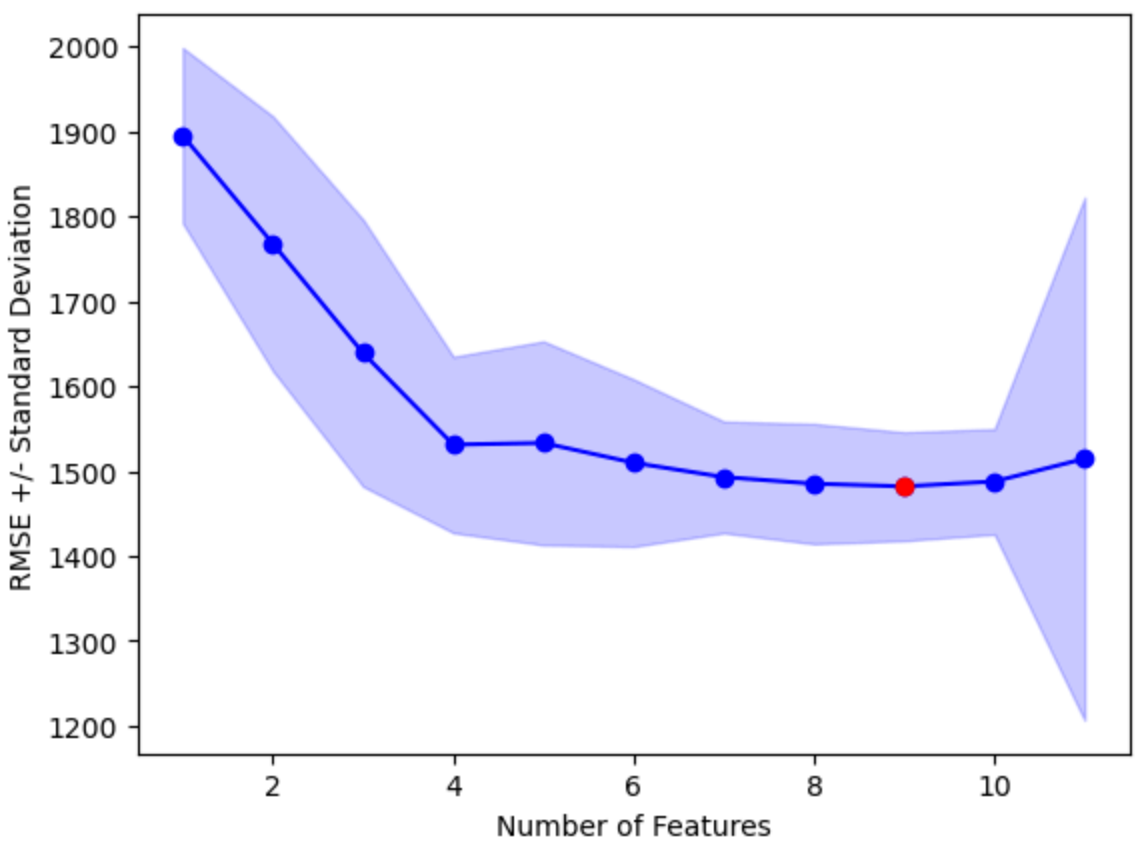
In [84]:

```
#VB67
# Performing backward sequential feature selection with Linear Regression
# Evaluating feature subsets from 1 to 11 features using 5-fold CV and RMSE

sfs_backward = SequentialFeatureSelector(model,
            k_features=(1, 11),
            forward=False, floating=False,
            cv=5, scoring='neg_root_mean_squared_error',
            n_jobs=-1)
sfs_backward = sfs_backward.fit(train_X, train_y)

# Starts with all features and removes them stepwise to find best subset
best_subset = sfs_backward.subsets_[1]
for v in sfs_backward.subsets_.values():
    if v['avg_score'] > best_subset['avg_score']:
        best_subset = v

# Printing best score, indices, and names of selected features
print(f"Best accuracy score: {- best_subset['avg_score']:.2f}")
print(f"Best subset (indices): {best_subset['feature_idx']}")
print(f"Best subset (corresponding names):\n{best_subset['feature_names']}")
```

```
Best accuracy score: 1482.00
Best subset (indices): (0, 1, 2, 4, 5, 6, 7, 9, 10)
Best subset (corresponding names):
('Age_08_04', 'KM', 'HP', 'Automatic', 'CC', 'Doors', 'Quarterly_Tax', 'Fuel
_Type_Diesel', 'Fuel_Type_Petrol')
```

In [85]:

```
#VB67
# Plotting Backward Selection Results - average RMSE +/- standard deviation

backward_subset = list(best_subset['feature_names'])

metric_dict = sfs_backward.get_metric_dict()

fig = plt.figure()
k_feat = sorted(metric_dict)
avg = [-metric_dict[k]['avg_score'] for k in k_feat]
best_idx = np.argmin(avg)

upper, lower = [], []
for k in k_feat:
    upper.append(-metric_dict[k]['avg_score'] + metric_dict[k]['std_dev'])
    lower.append(-metric_dict[k]['avg_score'] - metric_dict[k]['std_dev'])

plt.fill_between(k_feat, upper, lower, alpha=0.2, color='blue', lw=1)
plt.plot(k_feat, avg, color='blue', marker='o')
plt.plot(k_feat[best_idx], avg[best_idx], color='red', marker='o')
plt.ylabel('RMSE +/- Standard Deviation')
plt.xlabel('Number of Features')
plt.show()
```



In [24]:
```
#backward - Right to left
```

# Table 6.9 Stepwise Regression for Reducing Predictors in Toyota Corolla Example

In [62]:
```
#VB67
# Performing stepwise (forward-floating) feature selection with Linear Regre
# Evaluating feature subsets from 1 to 11 features using 5-fold CV and RMSE
# Features can be added or removed at each step to find the best subset
# Printing best score, indices, and names of selected features

sfs_stepwise = SequentialFeatureSelector(model,
            k_features=(1, 11),
            forward=True, floating=True,
            cv=5, scoring='neg_root_mean_squared_error',
            n_jobs=-1)

sfs_stepwise = sfs_stepwise.fit(train_X, train_y)

best_subset = sfs_stepwise.subsets_[1]
for v in sfs_stepwise.subsets_.values():
    if v['avg_score'] > best_subset['avg_score']:
        best_subset = v

print(f"Best accuracy score: {- best_subset['avg_score']:.2f}")
print(f"Best subset (indices): {best_subset['feature_idx']}")
print(f"Best subset (corresponding names):\n{best_subset['feature_names']}")
```

```
Best accuracy score: 1479.38
Best subset (indices): (0, 1, 2, 6, 8)
Best subset (corresponding names):
('Age_08_04', 'KM', 'HP', 'Doors', 'Weight')
```

In [86]:
```
#VB67
# Converting the best stepwise-selected features to a list and
# printing average CV scores for all evaluated subsets along with their feat

stepwise_subset = list(best_subset['feature_names'])
for _, results in sfs_stepwise.subsets_.items():
    print(results['avg_score'], results['feature_names'])
```

```
-1895.0266910419318 ('Age_08_04',)
-1697.9737764074853 ('Age_08_04', 'Weight')
-1551.8175978152856 ('Age_08_04', 'KM', 'Weight')
-1479.5701933226424 ('Age_08_04', 'KM', 'HP', 'Weight')
-1479.3835482399725 ('Age_08_04', 'KM', 'HP', 'Doors', 'Weight')
-1482.0817197199574 ('Age_08_04', 'KM', 'HP', 'Doors', 'Quarterly_Tax', 'Wei
ght')
-1486.7075301998175 ('Age_08_04', 'KM', 'HP', 'Met_Color', 'Automatic', 'Doo
rs', 'Weight')
-1492.1928294429476 ('Age_08_04', 'KM', 'HP', 'Met_Color', 'Automatic', 'Doo
rs', 'Quarterly_Tax', 'Weight')
-1481.9970846709712 ('Age_08_04', 'KM', 'HP', 'Automatic', 'CC', 'Doors', 'Q
uarterly_Tax', 'Fuel_Type_Diesel', 'Fuel_Type_Petrol')
-1487.7362826016172 ('Age_08_04', 'KM', 'HP', 'Met_Color', 'Automatic', 'C
C', 'Doors', 'Quarterly_Tax', 'Fuel_Type_Diesel', 'Fuel_Type_Petrol')
-1514.5058235816782 ('Age_08_04', 'KM', 'HP', 'Met_Color', 'Automatic', 'C
C', 'Doors', 'Quarterly_Tax', 'Weight', 'Fuel_Type_Diesel', 'Fuel_Type_Petro
l')
```

## Regularization (Shrinkage Models)

In Python, regularized linear regression can be run using methods Lasso and Ridge in sklearn.linear_model. The penalty parameter is set using the alpha argument (the default value is alpha=1; note that alpha=0 means no penalty and yields ordinary linear regression). The methods LassoCV, RidgeCV, and BayesianRidge offer the added benefit of automatically selecting the penalty parameter. LassoCV and RidgeCV use cross-validation to determine optimal values for the penalty parameter. A different approach is used in BayesianRidge which uses an iterative approach to derive the penalty parameter from the whole training set. Elastic net (which includes ridge and lasso regression as special cases) is implemented in the models ElasticNet and ElasticNetCV. Here, l1 can be set using the l1_ratio argument. The ElasticNetCV model automatically selects the optimal l1 and alpha parameters using cross-validation. Because none of these methods normalizes the data, we need to combine them with a StandardScaler. Here, we use a Pipeline to combine the two steps.

## Table 6.10 Lasso and Ridge Regression for Shrinking Predictors in Toyota Corolla Example

Lasso model

In [87]:
```
#VB67
# Performing Lasso regression with cross-validation to select optimal regula
# Standardize features before fitting using StandardScaler
# Using 5-fold CV to choose the best alpha from a range of values
# Fit model on training data, print chosen alpha and coefficients
# Evaluating model performance on holdout set using regression summary

lasso_cv = Pipeline([
```

```
    ['normalize', StandardScaler()],
    ['model', LassoCV(cv=5, alphas=10**(np.linspace(-1, 5, 50)))],
])
lasso_cv.fit(train_X, train_y)
print(f"Lasso-CV chosen regularization: {lasso_cv['model'].alpha_:.3f}")
print(f"Coefficients: {lasso_cv['model'].coef_.round(3)}")
mlba.regressionSummary(y_true=holdout_y, y_pred=lasso_cv.predict(holdout_X))
```

```
Lasso-CV chosen regularization: 152.642
Coefficients: [-2173.882  -630.588   418.233     0.        0.       -0.
-0.
    84.888   986.212    -0.        0.   ]

Regression statistics

Mean Error (ME) : 7.3500
Root Mean Squared Error (RMSE) : 1394.7003
Mean Absolute Error (MAE) : 1051.7327
Mean Percentage Error (MPE) : -1.6771
Mean Absolute Percentage Error (MAPE) : 9.4525
```

Ridge regression

Note that we don't specify the cross-validation folds for ridge regression. This is due to the fact that `RidgeCV` has an efficient and fast implementation that uses leave-one-out cross-validation by default.

In [37]:
```
#best model for holdout RMSE
```

In [88]:
```
#VB67
# Performing Ridge regression with cross-validation to select optimal regula
# Standardize features using StandardScaler before fitting
# Evaluating 50 alpha values using RidgeCV and store CV results
# Fit model on training data, print chosen alpha and coefficients
# Assessing model performance on holdout set using regression summary

ridge_cv = Pipeline([
    ['normalize', StandardScaler()],
    ['model', RidgeCV(alphas=10**(np.linspace(-1, 5, 50)), store_cv_results=
])
ridge_cv.fit(train_X, train_y)
print(f"Ridge-CV chosen regularization: {ridge_cv['model'].alpha_:.3f}")
print(f"Coefficients: {ridge_cv['model'].coef_.round(3)}")
mlba.regressionSummary(y_true=holdout_y, y_pred=ridge_cv.predict(holdout_X))
```

```
Ridge-CV chosen regularization: 49.417
Coefficients: [-1982.464  -752.963   626.569    26.732    19.29   -294.475
-20.054
   598.448  1019.766   271.592   386.263]

Regression statistics

Mean Error (ME) : 17.5822
Root Mean Squared Error (RMSE) : 1347.9356
Mean Absolute Error (MAE) : 1024.4671
Mean Percentage Error (MPE) : -1.3483
Mean Absolute Percentage Error (MAPE) : 9.0263
```

BayesianRidge regression

In [89]:
```
#VB67
# Fitting Bayesian Ridge regression with standardized features
# Standardize features using StandardScaler before fitting
# Fitting a model on training data and print coefficients
# Evaluating model performance on holdout set using regression summary

bayesianRidge = Pipeline([
    ['normalize', StandardScaler()],
    ['model', BayesianRidge()],
])
bayesianRidge.fit(train_X, train_y)
print(f"Coefficients: {bayesianRidge['model'].coef_.round(3)}")
mlba.regressionSummary(y_true=holdout_y, y_pred=bayesianRidge.predict(holdou
```

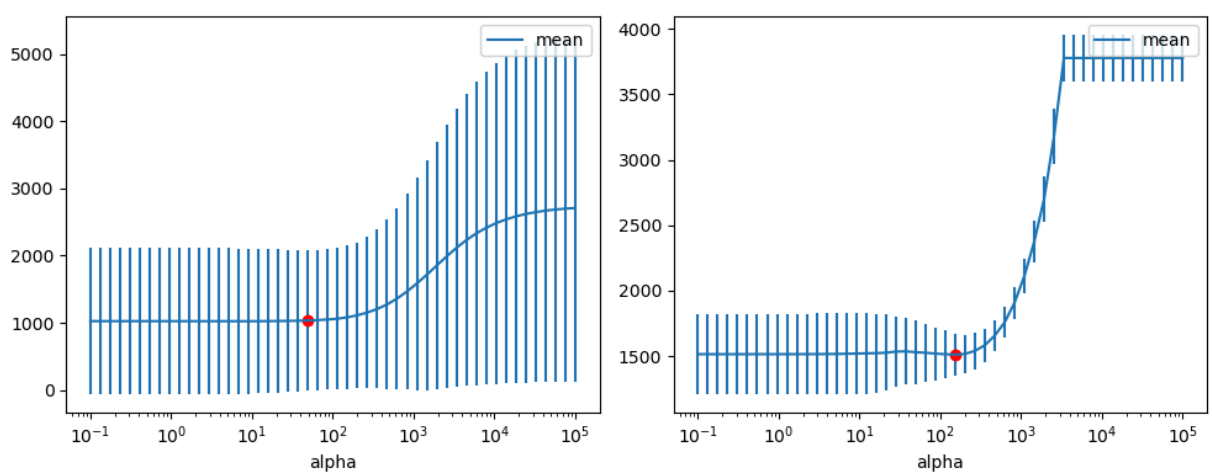
```
Coefficients: [-2169.536  -638.497   926.399    24.802    56.422  -715.318
-27.83
   756.072   958.937  1072.789   878.556]

Regression statistics

Mean Error (ME) : 0.3894
Root Mean Squared Error (RMSE) : 1350.0098
Mean Absolute Error (MAE) : 1025.4543
Mean Percentage Error (MPE) : -1.2010
Mean Absolute Percentage Error (MAPE) : 9.0411
```

# Figure 6.2 CROSS-VALIDATED RMSE OF TOYOTA PRICE PREDICTION MODELS AS A FUNCTION OF THE TUNING PARAMETER FOR THE RIDGE (LEFT) AND LASSO (RIGHT) REGRESSION. THE ERROR BARS ARE +/-STANDARD DEVIATION BASED ON THE RMSE VALUES FROM THE CROSS-VALIDATION FOLDS

For the lasso model, the optimal alpha value is determined using 5-fold cross-validation and selecting the parameter set that has the lowest RMSE value. For ridge regression, scikit-learn uses an efficient implementation of leave-one-out cross-validation.

Figure 6.2 shows the cross-validated RMSE as a function of the alpha parameter. The parameters for the best model are accessible with the alpha_ field. Here, both methods determined relatively small values for alpha. In general, this will not be the case. The potential range for alpha can be large, and Figure 6.2 clearly shows why an exponential sequence (on the x-axis) is required to explore the alpha parameter.

In [90]:
```
#VB67
# Ploting a regularization paths for Ridge and Lasso models
# Shows how cross-validated RMSE changes with different alpha values
# Highlighting the chosen alpha with a red marker

def plotRegularizationPath(ax, pipeline):
    model = pipeline['model']
    best_alpha = model.alpha_
    if isinstance(model, RidgeCV):
        df = pd.DataFrame({
            'alpha': model.alphas,
            'mean': np.sqrt(model.cv_results_).mean(axis=0),
            'std': np.sqrt(model.cv_results_).std(axis=0),
        })
    else:
        df = pd.DataFrame({
            'alpha': model.alphas_,
            'mean': np.sqrt(model.mse_path_).mean(axis=1),
            'std': np.sqrt(model.mse_path_).std(axis=1),
        })
# Using log scale for alpha axis and error bars for standard deviation
    best_score = df['mean'][df['alpha'] == best_alpha]
    df.plot(x='alpha', y='mean', yerr='std', logx=True, ax=ax)
    ax.plot(model.alpha_, best_score, 'ro')

# Subplots display Ridge (left) and Lasso (right) side by side
fig, axes = plt.subplots(ncols=2, figsize=(10, 4))
plotRegularizationPath(axes[0], ridge_cv)
plotRegularizationPath(axes[1], lasso_cv)
plt.tight_layout()
plt.show()
```



# Table 6.11 Comparing Performance of the Different Variable Selection Methods with the Full Model

In [91]:
```
#VB67
# Example: convert best features from ExhaustiveFeatureSelector to a list
efs_subset = list(efs.best_feature_names_)

# Now we can use efs_subset
print(efs_subset)

efs_model = LinearRegression()
efs_model.fit(train_X[efs_subset], train_y)

sfs_stepwise_model = LinearRegression()
sfs_stepwise_model.fit(train_X[stepwise_subset], train_y)

sfs_forward_model = LinearRegression()
sfs_forward_model.fit(train_X[forward_subset], train_y)

sfs_backward_model = LinearRegression()
sfs_backward_model.fit(train_X[backward_subset], train_y)

# Creating tuned models for lasso and ridge
lasso_opt = Pipeline([
    ['normalize', StandardScaler()],
    ['model', Lasso(alpha=lasso_cv['model'].alpha_)],
])

ridge_opt = Pipeline([
    ['normalize', StandardScaler()],
    ['model', Ridge(alpha=ridge_cv['model'].alpha_)],
])


def metrics(model, train_X, train_y, holdout_X, holdout_y, *, subset=None):
    if subset is not None:
        train_X = train_X[subset]
        holdout_X = holdout_X[subset]
        n_features = len(train_X.columns)
        model.fit(train_X, train_y)
    else:
        model.fit(train_X, train_y)
        if isinstance(model, Pipeline):
            n_features = sum(model['model'].coef_ != 0)
        else:
            n_features = sum(model.coef_ != 0)
    scoring = {'neg_RMSE': 'neg_root_mean_squared_error',
               'neg_MAE': 'neg_mean_absolute_error'}
    np.random.seed(123)
    scores = cross_validate(model, train_X, train_y, cv=5, scoring=scoring)

    train_pred = model.predict(train_X)
    holdout_pred = model.predict(holdout_X)
```

```
    return {
        'CV_RMSE': - scores['test_neg_RMSE'].mean(),
        'CV_MAE': - scores['test_neg_MAE'].mean(),
        'train_RMSE': np.sqrt(np.mean((train_y - train_pred)**2)),
        'train_MAE': np.mean(np.abs(train_y - train_pred)),
        'holdout_RMSE': np.sqrt(np.mean((holdout_y - holdout_pred)**2)),
        'holdout_MAE': np.mean(np.abs(holdout_y - holdout_pred)),
        'n_features': n_features,
    }
pd.DataFrame({
    'Full': metrics(car_lm, train_X, train_y, holdout_X, holdout_y),
    'Exhaustive': metrics(efs_model, train_X, train_y, holdout_X, holdout_y,
    'Forward': metrics(sfs_forward_model, train_X, train_y, holdout_X, holdc
    'Backward': metrics(sfs_backward_model, train_X, train_y, holdout_X, hol
    'Stepwise': metrics(sfs_stepwise_model, train_X, train_y, holdout_X, hol
    'Ridge': metrics(ridge_opt, train_X, train_y, holdout_X, holdout_y),
    'Lasso': metrics(lasso_opt, train_X, train_y, holdout_X, holdout_y),
}).transpose().round(0)
```

```
['Age_08_04', 'KM', 'HP', 'Doors', 'Weight']
```

Out[91]:

| | CV_RMSE | CV_MAE | train_RMSE | train_MAE | holdout_RMSE | holdout_MAE |
|---|---|---|---|---|---|---|
| **Full** | 1515.0 | 1053.0 | 1340.0 | 988.0 | 1355.0 | 1028.0 |
| **Exhaustive** | 1479.0 | 1068.0 | 1432.0 | 1035.0 | 1366.0 | 1027.0 |
| **Forward** | 1479.0 | 1068.0 | 1432.0 | 1035.0 | 1366.0 | 1027.0 |
| **Backward** | 1482.0 | 1124.0 | 1426.0 | 1081.0 | 1493.0 | 1133.0 |
| **Stepwise** | 1482.0 | 1124.0 | 1426.0 | 1081.0 | 1493.0 | 1133.0 |
| **Ridge** | 1487.0 | 1056.0 | 1375.0 | 1007.0 | 1348.0 | 1024.0 |
| **Lasso** | 1514.0 | 1080.0 | 1443.0 | 1039.0 | 1395.0 | 1052.0 |

# Using STATSMODELS

# Table 6.12 Linear Regression Model of Price vs. Car Attributes using STATSMODELS

An alternative to scikit-learn 's LinearRegression is method sm.ols in statsmodels. The latter produces a more extensive output of the statistical properties of the model, suitable for non-predictive tasks such as statistical inference. Table 6.12 shows the same model for price vs. car attributes from Table 6.3 run using sm.ols. For regularization, use statsmodels method OLS.fit_regularized. Set argument L1_wt =0 for ridge regression and L1_wt =1 for lasso.

In [92]:

```
#VB67
# running a linear regression of Price on the remaining 11 predictors in the
train_df = train_X.join(train_y)
```

```
predictors = train_X.columns
formula = 'Price ~ ' + ' + '.join(predictors)

car_lm = sm.ols(formula=formula, data=train_df).fit()
car_lm.summary()
```

Out[92]:

OLS Regression Results

| | | | |
|---|---|---|---|
| **Dep. Variable:** | Price | **R-squared:** | 0.874 |
| **Model:** | OLS | **Adj. R-squared:** | 0.872 |
| **Method:** | Least Squares | **F-statistic:** | 371.6 |
| **Date:** | Tue, 23 Sep 2025 | **Prob (F-statistic):** | 2.58e-256 |
| **Time:** | 15:27:33 | **Log-Likelihood:** | -5171.7 |
| **No. Observations:** | 600 | **AIC:** | 1.037e+04 |
| **Df Residuals:** | 588 | **BIC:** | 1.042e+04 |
| **Df Model:** | 11 | | |
| **Covariance Type:** | nonrobust | | |

| | coef | std err | t | P>\|t\| | [0.025 | 0.975] |
|---|---|---|---|---|---|---|
| **Intercept** | -1956.3598 | 1939.385 | -1.009 | 0.314 | -5765.325 | 1852.606 |
| **Fuel_Type_Diesel[T.True]** | 3875.7780 | 770.158 | 5.032 | 0.000 | 2363.182 | 5388.374 |
| **Fuel_Type_Petrol[T.True]** | 2897.5325 | 547.021 | 5.297 | 0.000 | 1823.180 | 3971.885 |
| **Age_08_04** | -130.7928 | 4.813 | -27.173 | 0.000 | -140.246 | -121.339 |
| **KM** | -0.0183 | 0.002 | -8.052 | 0.000 | -0.023 | -0.014 |
| **HP** | 64.8345 | 8.247 | 7.862 | 0.000 | 48.637 | 81.032 |
| **Met_Color** | 54.2823 | 120.096 | 0.452 | 0.651 | -181.587 | 290.152 |
| **Automatic** | 298.0097 | 276.402 | 1.078 | 0.281 | -244.847 | 840.866 |
| **CC** | -4.2514 | 0.842 | -5.048 | 0.000 | -5.905 | -2.597 |
| **Doors** | -25.8889 | 63.634 | -0.407 | 0.684 | -150.867 | 99.089 |
| **Quarterly_Tax** | 18.0132 | 2.670 | 6.748 | 0.000 | 12.770 | 23.256 |
| **Weight** | 15.5473 | 1.766 | 8.803 | 0.000 | 12.078 | 19.016 |

| | | | |
|---|---|---|---|
| **Omnibus:** | 97.514 | **Durbin-Watson:** | 1.924 |
| **Prob(Omnibus):** | 0.000 | **Jarque-Bera (JB):** | 778.538 |
| **Skew:** | -0.440 | **Prob(JB):** | 8.76e-170 |
| **Kurtosis:** | 8.511 | **Cond. No.** | 2.53e+06 |

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.53e+06. This might indicate that there are
strong multicollinearity or other numerical problems.

In [ ]:

```

```